

EDUCATION

University of Toronto

HBSc, Specialist in Data Science and Major in Computer Science

- CGPA: 3.79, **Dean's List** 2024 and 2025

Toronto, Ontario

Expected June 2026

EXPERIENCE

Full-Stack Developer → Tech Lead

Aug 2025 - Present

Ontario Association of Physics Teachers (OAPT), University of Toronto

Toronto, Ontario

- Promoted to **Tech Lead** after 5 months as Full-Stack Developer, owning end-to-end architecture and development of a contest platform designed for **660,000+ students** across **883+ Ontario schools**
- Architected full-stack web application using **React**, **Node.js/Express.js**, **PostgreSQL**, and **Prisma ORM**, designing a normalized database schema and **RESTful API** with **60+** endpoints
- Designed scalable backend architecture following **MVC** principles with clear separation of concerns across routes, controllers, services, and data access layers
- Modeled complex relational data (users, contests, questions, submissions, scoring) with **Prisma schema migrations** and foreign key constraints to enforce referential integrity
- Built secure authentication with **JWT tokens**, **bcrypt** password hashing, and **role-based access control (RBAC)** supporting student, teacher, and admin roles
- Implemented **middleware-based authorization** for granular permission control, including admin shadowban capabilities and route-level access guards
- Designed **RESTful API endpoints** following industry conventions, with consistent error handling, request validation, and structured JSON responses
- Used **PostgreSQL transactions** to ensure data integrity across multi-step operations, preventing partial writes and maintaining database consistency
- Designed and implemented the **contest editing feature** end-to-end, integrating **TipTap/ProseMirror** rich-text editing with **LaTeX** math rendering, drag-and-drop question ordering (**@dnd-kit**), and **PDF generation** via **Puppeteer** for printable contest formats
- Built **image upload pipeline** with **Multer**, enforcing file size limits and extension whitelisting (validating MIME type and extension) to prevent oversized uploads and unexpected file types from reaching storage
- Implemented responsive React UI with **Material-UI** centralized theming (**createTheme**), custom hooks (e.g. **withAuth** for route protection), and **Context API** for global state management
- Developed reusable **component library** (form inputs, modals, data tables) with consistent props API, reducing code duplication across pages
- Lead a team of **8 developers** alongside a senior developer using **Agile** principles, conducting GitHub PR code reviews and mentoring on Git workflows, RESTful API design, and React patterns
- Established team-wide development standards from scratch, including **PR templates**, issue tagging conventions, branching strategy, and code review expectations
- Restructured team workflows to distribute issue creation across all developers and built a **promotion path** for contributors to advance into PR reviewer roles
- Drive technical decision-making on architecture, technology stack, API design patterns, database schema evolution, and security implementations
- Authored technical documentation and codebase architecture guides to standardize development practices and streamline onboarding for new developers
- Built a **Discord webhook integration** pushing real-time notifications for issue and PR updates with manual developer tagging, eliminating missed communications and replacing unreliable email-based updates
- Coordinated development via **GitHub Issues**, **Trello**, and team communication on **Slack/Discord**, maintaining consistent sprint cadence
- Responded to a live security incident caused by a compromised npm package in the dependency chain, coordinating containment and implementing improved **dependency auditing** practices afterward

PROJECTS

Scriptorium (github)

Oct 2024 - Nov 2024

React, Node.js, Next.js, Tailwind CSS, SQLite, Prisma, Docker, nginx, AWS EC2

CSC309 Course Project (Team of 3)

- Built a **collaborative coding platform** supporting writing, executing, and sharing code across **10+ programming languages**, developed as a 3-person **CSC309** (Programming on the Web) team project
- Designed and implemented the **code template system**, enabling users to save, fork, tag, and share reusable code snippets, with cross-feature integration allowing templates to be embedded and referenced in blog posts
- Built the **code execution UI**, integrating real-time input/output handling with the backend execution service for a responsive in-browser coding experience
- Developed responsive frontend with **React**, **Next.js**, and **Tailwind CSS** for cross-device compatibility

- Owned **deployment infrastructure**, containerizing the application with **Docker** and configuring **nginx** as a reverse proxy in front of the Node.js backend on **AWS EC2**
- Collaborated on **RESTful API** design with **Node.js**, **Express**, and **Prisma ORM** backed by **SQLite**

Ant Colony Simulation ([github](#))

Jan 2026

C++, Ubisoft NEXT API

Ubisoft NEXT 2025-2026 Submission

- Solo submission to **Ubisoft NEXT 2025-2026** ("Simulation of Large Number of Entities" challenge), simulating two rival ant colonies with pheromone pathfinding and emergent swarm behaviors at **2000 entities** in real-time
- Built end-to-end in **C++** using only the framework's primitive draw calls (line, triangle, rectangle), self-learning **Entity Component System (ECS)** architecture from scratch over 3-4 weeks before 3 days of focused implementation
- Designed cache-friendly **ECS** with **Structure of Arrays (SoA)** memory layout (`std::tuple<std::vector<T>...>`) for contiguous component storage and CPU cache efficiency
- Implemented **O(1) component lookups** via per-entity bitmasks and bitwise AND, eliminating hash map overhead in the hot path
- Built per-type **cached entity lists**, rebuilt on add/remove instead of filtering 10,000 entities each frame
- Architected modular **system pipeline** (Ant, Spider, Collision, Drag, Movement, Render) with deterministic update ordering
- Engineered **uniform spatial grid** (32-unit cells) with templated **QueryEach** lambda interface for **zero-allocation** neighbor iteration
- Optimized proximity queries by batching them into a combined **NearbyQuery::QueryAll**, returning nearest spider, enemy ant, and food in one grid pass instead of three
- Designed **5-state finite state machine (FSM)** for agent behavior (WANDER, FOLLOW_TRAIL, FORAGE, ATTACK, FLEE) with per-frame context bundling references and queries to eliminate redundant lookups
- Implemented **3-layer pheromone system** (HOME per-team, FOOD shared, ALARM combat) producing emergent foraging trails, combat coordination, and team-specific homing
- Built **Monte Carlo cone sampling** for pheromone pathfinding (5 forward samples vs 25-neighbor gradient), preventing traffic jams and producing natural path variety
- Built **stuck detection** via rolling position history, forcing escape vectors when agents fail to make progress in dense crowds
- Pre-computed **sin/cos lookup tables** (32 entries) at startup, eliminating runtime trigonometry in the AI hot loop
- Implemented **staggered decay** with per-layer tick timers, distributing pheromone decay cost across frames instead of spiking each update
- Applied **frustum culling** via camera visibility checks before draw submission, skipping off-screen entities entirely
- Engineered **screen-space LOD** selecting render complexity (4-12 segments) by entity size and zoom, balancing visual fidelity against draw cost
- Designed **typed event buffer** for collisions (AntFood, AntSpider, FoodColony), separating detection from response across two passes to avoid modification-during-iteration bugs
- Built **Observer-pattern action system** mapping hardware input to named actions, decoupling scene logic from raw keycodes
- Implemented **debug visualization** with runtime toggles for individual pheromone layers, supporting iterative tuning of agent behavior

Salt, Grooves, and Garlic ([itch.io](#))

Sep 2025 - Present

Unity, C#

CSC404 Game Design Capstone (Team of 10)

- **2nd Best Game Overall** at **LevelUp 2026** showcase, a rhythm-action game built in **Unity (C#)** with a 10-person cross-functional team for the CSC404 game design capstone
- Architected core **game manager and state control systems** in C#, coordinating scene flow and gameplay subsystems across the full game loop, plus owned the **tutorial level** implementation (event scripting, dialogue system, audio transitions)

SKILLS

Programming Languages: C++, C#, JavaScript/TypeScript, Python, Java, R, C, MIPS Assembly

Frontend: React, Next.js, Material-UI (MUI), Tailwind CSS, HTML/CSS, Context API, Custom Hooks

Backend: Node.js, Express.js, RESTful APIs, JWT Authentication, RBAC, Middleware Design, MVC Architecture

Databases & ORMs: PostgreSQL, SQLite, Prisma ORM, Schema Design, Transactions, Migrations

DevOps & Infrastructure: Docker, nginx, AWS (EC2), Git/GitHub, Linux/Unix, Shell Scripting

Systems & Performance: Entity Component System (ECS), Cache-Friendly Memory Layout, Structure of Arrays (SoA), Spatial Partitioning, Lookup Tables, Zero-Allocation Patterns, FSM Design, Component-Based Design, Data-Driven Design

Development Practices: Code Review, Agile/Scrum, Iterative Development, Clean Architecture, OOP, Data Structures & Algorithms, Technical Documentation

Tools & Workflow: GitHub Issues, Trello, Slack/Discord, Visual Studio, VS Code, Unity Editor

Specialized Libraries: TipTap/ProseMirror, Puppeteer, Multer, LaTeX

AWARDS & ACTIVITIES

Awards: Indonesia Maju Scholarship

Teaching: Teaching Assistant for CSC309 (Programming on the Web), University of Toronto

Leadership: Led Recognized Study Group for CSC165 (Mathematical Expression and Reasoning for CS)

Languages: Indonesian (native), English (fluent)